Simulator 1: Customer Support Resolution Agent

Alignment: Domains 1, 2 and 5 · Runs on the Claude Agent SDK through your Claude login, against mock backends

Objective

Meet the support agent the exam describes, then measure it. Part A wires four MCP tools to a Claude Agent SDK loop, enforces the rules that prompts cannot guarantee with hooks, and runs twelve customer use cases through it; an evaluator scores every run and reports the first-contact resolution rate against the 80% target. Part B takes the same team to the next decision: proving a prompt template on a small evaluation set before 35,000 archived transcripts go through the Message Batches API, adding business-rule validation the schema cannot express, and packaging the review as a `/case-review` skill and a `claude -p` nightly check.

Scenario (official exam scenario 1)

"You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate."

Primary domains: Agentic Architecture & Orchestration, Tool Design & MCP Integration, Context Management & Reliability.

Prerequisites

- `uv` and a Claude login (`claude auth login`). Run `uv sync` once in the `labs` folder. No API key.
- Open this simulator in your browser with `uv run serve.py` from `labs`, or run the commands below from this simulator's folder.

© 2026 Tertiary Infotech Academy · UEN 201200696W

What is in the folder

File	Role
`backend_tools.py`	The mock Orderly backend as an in-process MCP server: `get_customer`, `lookup_order`, `recommend_refund`, `process_refund`, `escalate_to_human`, each with a description that says when to use it and when not to, and structured errors (`errorCategory`, `isRetryable`). `recommend_refund` takes a strict schema and then runs business rules the schema cannot express
`support_agent.py`	The agent: `query()` loop, system prompt, and three hooks (prerequisite gate, $500 refund limit, PostToolUse normaliser)
`prompts/system.md`	Policy, escalation criteria and few-shot examples; `system-noguard.md` is the same prompt without the $500 rule
`cases/CS-01.json` … `CS-13.json`	Use-case templates: customer contact, message (or `turns` for a multi-turn case), expected disposition, tool expectations and a rubric
`cases/UC-*.json`	Practice-test use cases: the exam situation, the correct option and why, the command to run here, what to observe, rubric
`triage.py`, `triage.schema.json`, `prompts/triage_template.md` (+ `_v0`)	Part B: prompt template with `{{transcript}}` placeholders, pilot through `output_format`, business-rule validation and retry, Message Batches by `custom_id`
`fixtures/transcripts/T-001.txt` … `T-010.txt`, `labels.json`	The evaluation set: ten archived transcripts with labelled disposition and refund total
`.claude/skills/case-review/SKILL.md`	The `/case-review [case-id]` skill: `context: fork`, `allowed-tools: Read, Grep`, `argument-hint`
`nightly_check.py`	CI-style review of every run, with a JSON schema and a non-zero exit on any FAIL, through the Agent SDK
`evaluate.py`	Deterministic scoring from the tool trace, optional independent LLM judge, FCR rate
`fixtures/backend.json`	Customers, orders, policy limits; one order is flaky on first lookup, two customers share a name
`verify.py`	Acceptance checks over `evidence/`

Guided tour

1. Read the four tool descriptions in `backend_tools.py`. Each one names its input format, when to use it, and the neighbour it must not be confused with. This is the primary routing signal the model has.

```
sed -n '1,120p' backend_tools.py
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

2. Read the hooks in `support_agent.py`. The prerequisite gate denies `lookup_order` and `process_refund` until `get_customer` has returned an id; the validation gate denies `process_refund` until `recommend_refund` has returned `validated: true` for the same order and amount; the limit hook denies any refund above $500 and tells the model to escalate; the PostToolUse hook rewrites unix timestamps and numeric status codes before Claude reads them.

```
grep -n "permissionDecision\|updatedToolOutput\|verified_customer_id" support_agent.py
```

3. Run the first use case and watch the loop: `get_customer`, then `lookup_order`, then `process_refund`, then `end_turn`.

```
uv run support_agent.py --case CS-01
```

4. Run the over-limit case twice. With the normal prompt the model escalates on its own. With the prompt that omits the rule, the model attempts the refund and the hook stops it: that is the difference between probabilistic and deterministic compliance.

```
uv run support_agent.py --case CS-02
uv run support_agent.py --case CS-02 --prompt prompts/system-noguard.md --tag noguard
```

5. Run every use case. Cases cover a volunteered order number (gate), two concerns in one message, an explicit request for a human, a transient timeout, a return, a policy gap (competitor price match), an ambiguous name, a frustrated but routine customer, a refund the tool refuses with `isError` (closed return window), a three-turn conversation across three orders where the case-facts block has to keep the amounts straight, and a partial refund with store credit where the recommendation is schema-valid yet can still be wrong in meaning.

Post-validation, in detail (practice test Q4): `recommend_refund` accepts a strict schema, so shape and types are always right. Its handler then checks the invariants: every sku is on the order and not refunded above its price, `shipping_refund` never exceeds the shipping fee, `merchandise_credit` is store credit and never counts as cash, and `refund_total` equals item adjustments plus shipping refund. A failing recommendation comes back as `isError` with the named failures, the model corrects it, and only a validated total can reach `process_refund`. Run CS-13 with and without the gate:

```
uv run backend_tools.py --demo
uv run support_agent.py --case CS-13
uv run support_agent.py --case CS-13 --no-validation --tag novalidation
```

The `--demo` run feeds the validator four schema-valid recommendations without calling the model: store credit typed into `shipping_refund`, lines that do not add up, a sku that is not on the order, and the correct one. Read the failure text; that is exactly what the agent receives as `isError` content.

```
uv run support_agent.py --all
```

6. Score the runs. The deterministic layer checks the trace; `--judge` adds an independent Claude instance that never saw the conversation. Read the FCR line at the end.

```
uv run evaluate.py --judge
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

7. Switch the prerequisite gate off and rerun. Compare how often the model still verifies identity first. Prompt wording alone has a non-zero failure rate; the gate makes it zero.

```
uv run support_agent.py --all --no-gate --tag nogate
uv run evaluate.py --tag nogate
```

8. Write your own use case. Copy `cases/CS-07.json`, change the message, the expected disposition and the rubric, then run it. Good candidates: a refund request for an order that belongs to another customer, a customer who repeats the request for a human after you offered to help, a damaged item with no photo.

```
cp cases/CS-07.json cases/CS-11.json && uv run support_agent.py --case CS-11 && uv run
    evaluate.py
```

9. Part B. Run the weak template, then the real one, over the ten-transcript evaluation set. The weak template has no placeholders for policy or id and no refund rules; watch first-pass success and the business-rule errors that come back (store credit in `shipping_refund`, totals that do not add up). Only the real template reports `ready for the overnight batch`.

```
uv run triage.py --pilot --template prompts/triage_template_v0.md
uv run triage.py --pilot
```

10. Read the validator. Schema validation already passed on every record; these checks are about meaning. Then send the same ten transcripts through the batch workflow, simulated locally through the Agent SDK, and read the note on rows that stop at `stop_reason: tool_use`.

```
grep -n "def business_rules" -A 16 triage.py
uv run triage.py --batch
```

11. Invoke the `/case-review` skill on the over-limit case. It runs in a forked context, so the evidence JSON never lands in your main session, and it needs no permission prompts for Read or Grep.

```
cat .claude/skills/case-review/SKILL.md
uv run case_review.py CS-02
```

12. Run the nightly check the way CI would: print mode, JSON output, a schema, and an exit code the pipeline can act on.

```
./nightly_check.py; echo "exit $?"
uv run python -m json.tool evidence/nightly_check.json | head -40
```

13. Run the verifier and save its output.

```
uv run verify.py | tee evidence/verification.txt
```

Labs that run on this simulator

Open this simulator in your browser: from the `labs` folder run `uv run serve.py`, then pick Simulator 1. In Claude Code, type `/lab` and the simulator number. A simulator is the system. A lab is one exam situation you run on it. Run any of these from the `labs` folder with `uv run run_lab.py <lab>`; each has its own PDF under its topic folder. An asterisk marks a lab run in class.

© 2026 Tertiary Infotech Academy · UEN 201200696W

Lab	Task	Source	What it proves	Command
1*	1.1	Task statement	Watch one refund go round the loop	`uv run support_agent.py --case CS-01`
9*	1.4	Guide Q1	12% of cases skip get_customer: fix it with a gate, not a sentence	`uv run support_agent.py --case CS-03`
12*	1.5	Task statement	The prompt forgot the $500 rule. The hook did not.	`uv run support_agent.py --case CS-02`
19*	2.2	Practice Q29	The refund failed. The customer was told it worked.	`uv run support_agent.py --case CS-11`
20	2.2	Task statement	Retry the timeout once, then answer from the real record	`uv run support_agent.py --case CS-06`
29*	3.2	Practice Q16	Keep a noisy case review out of your main session	`uv run case_review.py CS-02`
43*	4.4	Practice Q4	Valid JSON, wrong refund: add business rules	`uv run backend_tools.py --demo`
45*	4.5	Practice Q1	Prove the template before 35,000 transcripts	`uv run triage.py --pilot --template prompts/triage_template_v0.md`
46	4.5	Practice Q26	Client tools can't run inside a batch	`uv run triage.py --batch`
51*	5.1	Practice Q40	Refund amount jumped to the wrong order	`uv run support_agent.py --case CS-12`
54*	5.2	Guide Q3	55% first-contact resolution: calibrate with criteria and examples	`uv run support_agent.py --case CS-05 --case CS-08 --case CS-09 --case CS-10`

Expected evidence

- `evidence/runs/CS-*.json`: every tool call with input, raw result, normalised result, hook decision, the reply, stop reason and cost.
- `evidence/evaluation.json`: per-case checks, judge scores and the first-contact resolution rate.
- `evidence/runs/CS-02-noguard.json`: the hook denying a $780 refund that the prompt did not prevent.
- `evidence/triage-triage_template_v0.json` and `evidence/triage-triage_template.json`: first-pass success, business-rule errors, retries, accuracy against the labels, and the `ready_for_batch` flag.
- `evidence/case-review-CS-02.txt` and `evidence/nightly_check.json`: the skill's review and the CI check's structured verdicts with cost.

Acceptance checks

- `get_customer` precedes every successful order operation in every run.
- The $780 refund is never issued; the handoff carries customer_id, root cause, recommended action and refund amount.
- The explicit human request is escalated without a single `lookup_order`.
- The transient error is retried once and the final answer uses the real order record.
- The ambiguous name produces a clarifying question, not a guessed account.

© 2026 Tertiary Infotech Academy · UEN 201200696W

- The closed-window refund is never issued and the reply never claims it was.
- After three turns the only refund is $45 on ORD-5503 and every reply carried a CASE FACTS block.
- On CS-13 the cash refund is $105 for the lamp and mat, the $8 shipping is store credit, and `recommend_refund` validated before `process_refund` ran.
- FCR across the twelve cases is at least 80%.
- The real template reaches 100% first-pass success on the evaluation set; the weak one does not.

Exam link

Task statements 1.1 (loop on stop_reason), 1.4 (programmatic prerequisites, multi-concern decomposition, structured handoff), 1.5 (PostToolUse normalisation, tool-call interception), 2.1 (tool descriptions as routing contracts), 2.2 (structured errors, retryable or not), 3.2 (skills with context: fork, allowed-tools, argument-hint), 3.6 (claude -p with --output-format json and --json-schema), 4.3 and 4.4 (schema shape vs semantic validation, retry with error feedback), 4.5 (Message Batches, custom_id, what cannot run inside a batch), 5.1 (case-facts block), 5.2 (escalation criteria, honouring a human request, policy gaps, ambiguous matches).

© 2026 Tertiary Infotech Academy · UEN 201200696W